PRASHANT GROUP

Workforce Recruitment & Management System

Comprehensive Technical Dossier: QA Testing Strategy, Application Security, Production Deployment & Cloud Observability

Target Audience:	QA / Testing Engineers, Security & Penetration Testers, DevOps & SRE, Tech Leads
System Architecture:	FastAPI (ASGI) + SQLAlchemy 2.0 ORM + SQLite/PostgreSQL + Jinja2 + ReportLab
Application Version:	v2.4.0 (Enterprise Workforce Edition)
Document Classification:	Confidential / Restricted Technical Specification
Generated Date:	October 2026
Corporate Office:	Office No 217, Metropol Building, Dange Chowk, Thergaon, Pune - 411033
Website & Contact:	www.prashantgroupindia.in | +91 9922475940

Executive Purpose: *This document acts as the definitive single-source-of-truth for testing, securing, deploying, and monitoring the Prashant Group Workforce Recruitment & Management System. The system coordinates multi-stage industrial staffing workflows including candidate talent pools, job applications, skill evaluation interviews, client deployments, worker ID card generation, company-grouped workforce oversight, GPS-verified mobile supervisor attendance check-ins, and a 3-tier complaint escalation matrix.*

Document Structure & Navigation

Section 1: System Overview & Architecture	Role hierarchy, workflow mechanics, and software stack components
Section 2: Comprehensive QA Testing Strategy	Unit, integration, E2E, GPS mocking, concurrency, and load testing
Section 3: Security & Penetration Testing Blueprint	JWT auth, RBAC authorization, SQLi/XSS defense, OWASP Top 10 matrix
Section 4: Production Deployment Architecture	Docker containerization, Nginx reverse proxy, SSL/TLS, and PostgreSQL migration
Section 5: Observability, Logging & SRE Operations	Health metrics, structured logging, GPS audit trail, disaster recovery
Section 6: Database Schema & API Route Inventory	Relational model definitions and RESTful API endpoints reference

1. System Architecture & Technical Stack

The Prashant Group platform is built on modern asynchronous Python infrastructure designed for high throughput, low latency, and robust relational data management across industrial client locations.

Component	Technology / Framework	Key Role in Application
Backend Core	FastAPI (Python 3.10+)	Asynchronous ASGI REST API framework with Pydantic validation
Web Server	Uvicorn / Gunicorn	High performance ASGI web server handling HTTP requests and keep-alive
Database ORM	SQLAlchemy 2.0 (Declarative)	Type-safe object relational mapping with parameterized query protection
Database Engines	SQLite (Dev) / PostgreSQL 15+ (Prod)	Relational storage with foreign keys, indexes, and ACID compliance
Auth & Security	python-jose + passlib (bcrypt)	HMAC-SHA256 JWT tokens and salted bcrypt password hashing
Template Engine	Jinja2 SSR	Server-side rendered dynamic portals (Admin, Candidate, Supervisor)
Document Engine	ReportLab 5.0	Automated PDF generation for official offer letters, slips, and ID cards
Client-Side Geolocation	HTML5 Geolocation API	Browser-based GPS coordinate capture for on-site supervisor check-ins

System Roles & Authorization Matrix

The system strictly segregates operations across 4 primary stakeholder personas:

1. **SuperAdmin / HR Admin:** Global system access. Manages company clients, job postings, candidate evaluations, worker deployments, company-wise workforce rosters, GPS attendance logs, and resolving escalated worker complaints.
2. **Field Supervisor:** Mobile-enabled supervisor portal. Manages assigned client companies, browses deployed workers, submits daily worker attendance with mandatory real-time GPS coordinate logging, and reviews local worker complaints.
3. **Candidate / Applicant:** External user portal. Browses active industrial jobs, submits personal and trade qualifications, uploads identification documents, and tracks interview/offer statuses.
4. **Active Worker:** Deployed industrial workforce. Accesses worker portal, views monthly attendance, downloads digital ID cards, and files workplace grievances/complaints.

2. Comprehensive QA & Testing Strategy

The testing team should structure verification across a multi-tiered test pyramid to guarantee zero regression, complete data integrity, and strict boundary adherence before production rollout.

2.1 Test Pyramid Distribution & Responsibilities

Tier	Scope	Tooling	Target Coverage / Acceptance Criteria
Unit Tests	Models, Pydantic schemas, password hashing, token validation, PDF generators	pytest, pytest-mock	90%+ code coverage for business logic and calculation algorithms
API Integration	Endpoint response codes, validation errors, DB commits, foreign keys	FastAPI TestClient, HTTPX, FactoryBoy	100% endpoint coverage with both valid and boundary invalid payloads
Security / RBAC	Privilege escalation, unauthenticated access, IDOR on worker records	pytest-security, OWASP ZAP, Postman	Zero high/critical security findings; zero 403 bypass loopholes
Hardware / GPS	Supervisor attendance geolocation accuracy, spoofing, null-location handling	Playwright GPS emulation, TestClient	Coordinate validation: latitude (-90 to +90), longitude (-180 to +180)
Load / Stress	Simultaneous morning check-ins by 200+ supervisors at shift start	Locust, k6	< 350ms p95 response time at 500 concurrent requests/second

2.2 Critical E2E Test Cases & Acceptance Scenarios

Test Case TC-01: Candidate Separation vs Worker Rostering

Action: Register a candidate; verify candidate appears in /dashboard/admin/candidates. Deploy the candidate to a client company (e.g. Tata Motors); verify the candidate converts to a Worker and immediately appears in /dashboard/admin/workers under the Tata Motors company card and is removed from the active candidate pool.

Test Case TC-02: Company-Wise ID Card Rendering

Action: In the Admin Workers page, select Tata Motors. Verify that only Tata Motors personnel are listed in ID-card layout with photo, employee ID, trade designation, emergency contact, and deployment date.

Test Case TC-03: GPS-Tagged Attendance Verification

Action: Supervisor logs in, navigates to attendance tab, grants browser location. Submit attendance for 25 workers. Inspect database record to ensure supervisor_lat and supervisor_lng match client premises within acceptable tolerance (< 200m). Test denial of location permission: system must alert supervisor that GPS capture is required.

Test Case TC-04: 3-Tier Worker Complaint Lifecycle

Action: Worker submits grievance via portal. Verify supervisor sees complaint in supervisor dashboard. Supervisor enters an action remark and submits. Verify admin sees complaint status update in real-time, displays supervisor action remark, and admin can post final resolution.

3. Security Architecture & Penetration Testing Blueprint

Industrial workforce management systems store sensitive personally identifiable information (PII) including Aadhar/PAN numbers, bank account details, biometric/photo records, and geo-tracking logs. Security must follow defense-in-depth principles across transport, application, and database layers.

3.1 Authentication & Session Management

- **Algorithm & Secret Management:** JWT signed via HMAC-SHA256 (HS256). The secret key must be extracted exclusively from environment variables (.env) and must be at least 256 bits (32+ cryptographically random bytes) in production.
- **Token Expiration:** Access tokens set to expire after 120 minutes. Implement refresh token rotation with single-use revocation in Redis.
- **Password Security:** Uses passlib's bcrypt hashing scheme with a default cost factor of 12 rounds. Passwords are never stored in plaintext or logged in server telemetry.
- **Cookie Security Flags:** Auth cookies must enforce HttpOnly=True, Secure=True, and SameSite=Lax to prevent XSS credential exfiltration and CSRF attacks.

3.2 OWASP Top 10 Defense Matrix for Testing Team

Vulnerability	Threat Scenario	Built-In Countermeasure	QA Test Action
A01: Broken Access Control	Worker tampers URL ID to view another worker's complaints or salary	FastAPI dependency checks current user ID against requested resource ID	Attempt IDOR via curl with Worker A token on Worker B resource
A02: Cryptographic Failures	Exposure of passwords or PII in logs or HTTP wire	Bcrypt salting, HTTPS enforced via HSTS headers, PII scrubbed in logs	Inspect access logs; check SSL Labs score (target A+ rating)
A03: Injection Attacks	SQL injection in search parameters (e.g. searching candidate name)	SQLAlchemy 2.0 uses strictly parameterized statements. Zero raw string SQL	Run sqlmap or SQLi payloads (' OR 1=1--) against all search fields
A04: Insecure Design	Supervisor falsifies attendance location by tampering browser GPS	Backend validates coordinate bounds and cross-checks client IP geo-range	Transmit latitude 95.0 or 0,0 coordinates; verify 422 Unprocessable rejection
A05: Security Misconfig	Exposing FastAPI Swagger UI (`/docs`) in production	Disable docs when ENVIRONMENT=production in config.py	Verify `/docs` and `/redoc` return 404 in production mode
A07: Identification Failures	Brute-forcing admin login credentials	Rate-limiting middleware (slowapi) restricts to 5 failed attempts per IP	Send 20 rapid invalid login requests; assert HTTP 429 Too Many Requests

3.3 File Upload & Storage Hardening

Candidate resumes, certificates, and worker photos uploaded to /uploads must be strictly quarantined:

- **Whitelisted MIME Types:** Only image/jpeg, image/png, application/pdf accepted.
- **File Size Limit:** Hard limit of 10MB enforced via HTTP content-length header and streaming chunk verification.
- **Path Traversal Prevention:** Filenames sanitized with UUID prefix (e.g. uuid4()_filename.pdf) to prevent directory climbing attacks.
- **Static Serving Restriction:** Upload directory is configured with X-Content-Type-Options: nosniff and executable extensions (.py, .sh, .php) cannot be executed.

4. Production Deployment & Cloud Infrastructure

To achieve 99.9% uptime, seamless horizontal scalability, and automated rollbacks, the application is deployed using containerized microservices behind a high-concurrency Nginx reverse proxy.

4.1 Architecture Topology

Client Browsers → [Port 443 HTTPS / TLS 1.3] → Nginx Reverse Proxy (Static file caching & SSL offloading) → Gunicorn Process Manager (4 Uvicorn Workers on unix socket) → PostgreSQL 15 Database Cluster (Primary-Replica).

4.2 Production Dockerfile Specification

```
# Multi-stage production build
FROM python:3.11-slim as builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends build-essential libpq-dev
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

FROM python:3.11-slim as runner
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends libpq5 curl && rm -rf /var/lib/apt/lists/*
COPY --from=builder /root/.local /root/.local
ENV PATH=/root/.local/bin:$PATH
COPY backend/ ./backend/
COPY frontend/ ./frontend/
RUN mkdir -p uploads/letters
EXPOSE 8000
CMD ["gunicorn", "backend.main:app", "-w", "4", "-k", "uvicorn.workers.UvicornWorker", "-b", "0.0.0.0:8000"
]
```

4.3 Database Migration & Enterprise PostgreSQL Transition

Development currently uses SQLite (prashant_group.db). For production staging, migrate to PostgreSQL 15:

1. **Connection String:** Set DATABASE_URL=postgresql+psycopg2://user:password@db-host:5432/prashant_prod in .env.
2. **Connection Pool Tuning:** Configure SQLAlchemy pool with pool_size=20, max_overflow=10, and pool_recycle=1800.
3. **Schema Versioning:** Alembic migrations track every schema evolution: alembic revision --autogenerate -m 'version_tag' followed by alembic upgrade head during CI/CD deploy.

4.4 CI/CD Automated Pipeline Stages (GitHub Actions)

Pipeline Stage	Triggers	Execution Commands & Quality Gates
1. Lint & Format	Pull Request	ruff check . and black --check . (0 warnings allowed)
2. Unit & API Tests	Pull Request	pytest --cov=backend tests/ (Must exceed 85% branch coverage)
3. Security Scan	Main Push	bandit -r backend/ and trivy fs . for vulnerable dependencies
4. Container Build	Tag / Release	Docker multi-stage build, push to AWS ECR / GitHub Container Registry
5. Rolling Deploy	Approved Tag	SSH deploy or Kubernetes helm upgrade with zero-downtime healthcheck

5. Observability, Logging & SRE Operations

A reliable production system requires end-to-end visibility into response latencies, error frequencies, and physical field anomalies such as supervisor geo-distance discrepancies.

5.1 Structured Logging Architecture

All application logs are formatted in JSON structure to support effortless ingestion into ELK Stack, Datadog, or AWS CloudWatch:

```
{
  "timestamp": "2026-10-18T01:15:30.124Z",
  "level": "INFO",
  "module": "api.v1.supervisor",
  "event": "ATTENDANCE_SUBMITTED",
  "supervisor_id": 4,
  "company": "Tata Motors",
  "workers_marked": 28,
  "geo_lat": 18.6298,
  "geo_lng": 73.8123,
  "geo_accuracy_meters": 12.5,
  "response_time_ms": 48.2
}
```

5.2 Health Checks & Metric Alerts

Metric Category	Threshold / Alert Rule	Target Channel	Remediation SOP
API Availability	Endpoint /health fails 2 consecutive probes	PagerDuty / Urgent SMS	Auto-restart worker container; check DB connection health
Error Rate	5xx Server Errors > 1.5% of requests over 5 min	Slack #devops-alerts	Examine stack traces in Sentry/CloudWatch; roll back deploy
Database Latency	SQL query execution time > 800ms	Slack #db-admins	Inspect slow query log; verify table index health & vacuum DB
Disk Capacity	Uploads volume exceeds 80% capacity	Email to IT Admin	Trigger archival script moving documents older than 180d to S3
GPS Anomaly	Supervisor marks attendance > 5km from registered client	Admin Dashboard Flag	Auto-flag record as 'Unverified Location' for admin audit

5.3 Backup & Disaster Recovery Schedule

- **Automated Database Dumps:** Daily pg_dump executed at 02:00 AM IST, encrypted with AES-256 and uploaded to an offsite S3 bucket with 30-day lifecycle expiration.
- **Point-in-Time Recovery (PITR):** PostgreSQL Write-Ahead Logs (WAL) continuously streamed to backup storage allowing recovery to any point within the last 7 days.
- **RTO & RPO Objectives:** Recovery Time Objective (RTO) < 30 minutes; Recovery Point Objective (RPO) < 5 minutes.

6. Database Relational Model & API Endpoint Inventory

This section documents the primary data models and active RESTful API endpoints for validation by QA test engineers.

6.1 Core Database Tables & Relations

Table Name	Primary Key	Key Foreign Keys & Indexed Columns	Business Entity Description
users	id (Integer)	email (Unique), role (admin, candidate, supervisor)	System authentication accounts and access credentials
candidates	id (Integer)	user_id → users.id, phone, status	Job applicants with trade, education, and address details
jobs	id (Integer)	company_name, designation, is_active	Open industrial manpower job vacancies
applications	id (Integer)	candidate_id → candidates.id, job_id → jobs.id	Application lifecycle tracking from applied to deployed
interviews	id (Integer)	application_id → applications.id, status	Technical and physical vetting interview scorecards
deployments	id (Integer)	candidate_id, company_name, supervisor_id	Official deployment orders allocating workers to client plants
workers	id (Integer)	worker_id (Unique), company_name, is_active	Active on-site manpower roster linked to deployment records
attendance	id (Integer)	worker_id, supervisor_id, date, status, lat, lng	Daily check-in logs with supervisor GPS coordinates
complaints	id (Integer)	worker_id, company_name, status, created_at	Workplace grievances with supervisor remarks and admin resolution

6.2 Key API Endpoint Inventory for QA Validation

Method & Route	Role Required	Description & Expected Response
POST /api/v1/auth/login	Public	Authenticates user; returns JWT token and user profile object
GET /api/v1/admin/candidates	Admin	Returns candidate applications filtered by status (excludes active workers)
GET /api/v1/admin/workers	Admin	Returns workers grouped by company with ID-card card layout data
GET /api/v1/admin/complaints	Admin	Returns all worker complaints showing supervisor actions and statuses
POST /api/v1/admin/complaints/{id}/reply	Admin	Records admin resolution and closes or escalates grievance
GET /api/v1/supervisor/workers	Supervisor	Returns workers under supervisor's assigned companies
POST /api/v1/supervisor/attendance	Supervisor	Records batch attendance payload with required GPS coordinates (lat, lng)
POST /api/v1/supervisor/complaints/{id}/action	Supervisor	Submits supervisor investigation remark and action status
POST /api/v1/worker/complaints	Worker	Worker files a new workplace complaint or safety concern

Engineering & QA Approval Sign-Off:

Lead System Architect: _____ QA / Testing Lead: _____

Security & DevOps Lead: _____ Director, Prashant Group: _____